

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет

Практическая/Лабораторная работа

Выполнил:

Кутуков Даниил

К3339

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

1. Цель работы

Освоить полный цикл разработки backend-приложения с инфраструктурой в Docker и реляционной базой данных на реальном проекте: проектирование схемы предметной области, контейнеризация окружения, применение миграций, инициализация данных при старте, реализация REST API с версионированием, настройка аутентификации и базовой проверки работоспособности сервиса.

2. Задачи

- Спроектировать модель данных для домена управления объектами недвижимости и аналитикой (пользователи, здания, арендаторы, договоры, начисления/биллинг, чаты/сообщения, вложения, настройки, сервисы/аналитики, логи действий администратора).
- Развернуть инфраструктурные сервисы в Docker: PostgreSQL, Redis, RabbitMQ, MinIO (S3-совместимое хранилище) и gateway-прокси.
- Описывать и поддерживать схему БД через SQLAlchemy + Alembic и автоматически применять миграции при старте приложения.
- Реализовать REST API на FastAPI с версионированием `/api/v1` и health-endpoint'ом.
- Настроить аутентификацию по JWT и авторизацию через HTTP Bearer (включая refresh-токены).
- Реализовать фоновые/вспомогательные сервисы проекта (`ml`, `scheduler`) и интеграции через RabbitMQ.
- Подготовить базовые сценарии проверки API (ручная проверка по health и интеграционные тесты pytest для ключевых потоков регистрации/активации и работы с чатами).

3. Краткое описание реализованного сервиса

Реализован backend системы ****Pulse Project**** для управления данными по зданиям, арендаторам и финансовым/эксплуатационным показателям, а также коммуникациями внутри системы:

- Пользователи (`users`) имеют роли и статусы (активность/верификация e-mail) и участвуют в бизнес-процессах (регистрация, активация, назначение зданий).
- Сущности предметной области: здания (`buildings`), арендаторы (`renters`), договоры (`contracts`), начисления/биллинг (`billing`) и категории биллинга (`billing\_categories`).

- Для поддержки аналитики предусмотрены сущности ``services``, ``analytics``, ``analytics_result`` и связанные данные датчиков/показаний (``sensors``, ``sensor_readings``).
- Для коммуникации реализованы чаты и сообщения (``chats``, ``chat_members``, ``messages``), а для обмена файлами — вложения (``attachments``) с хранением в S3-совместимом хранилище.
- Действия администратора фиксируются в ``admin_actions_logs``, глобальные параметры — в ``settings``.

4. Архитектурные и технологические решения

- Backend: **FastAPI**, запуск через ``uvicorn`` (dev/обычный режим) и ``gunicorn`` (в production-режиме на non-windows).
- Данные: **PostgreSQL 14**; ORM и миграции — **SQLAlchemy 2** + **Alembic** (асинхронный engine через ``asyncpg``).
- Кэш и хранение служебных данных: **Redis**.
- Очереди/интеграции: **RabbitMQ** (aio-pika).
- Объектное хранилище: **MinIO** (S3 API), доступ через ``aioboto3``/``boto3``.
- Декомпозиция на сервисы: контейнеры ``backend``, ``ml``, ``backend-scheduler`` + инфраструктура (postgres/redis/rabbitmq/minio) и ``gateway`` (nginx) для проксирования запросов к ``/api/v1`` и ``/api/v1/ws``.
- Диагностика: healthcheck эндпоинт ``GET /api/v1/health`` (используется docker healthcheck).

5. Развертывание базы данных и окружения

5.1 Контейнеризация

В ``docker-compose.yml`` описаны сервисы:

- ``postgres`` (PostgreSQL 14, volume ``postgres_data``),
- ``redis`` (с паролем, AOF, ограничение памяти),
- ``rabbitmq`` (management-версия),
- ``minio`` + ``minio-init`` (создание bucket и пользователя с политикой доступа),
- ``backend`` (API),

- ``backend-scheduler`` (планировщик/фоновые задачи),
- ``ml`` (ML/обработчики задач),
- ``gateway`` (nginx, проксирование ``/api/v1`` и websocket'ов ``/api/v1/ws``).

Для локальной разработки предусмотрен ``docker-compose.local.yml``, который переопределяет образы сервисов на локально собранные (``backend:latest``, ``ml:latest``, ``gateway:latest``).

Для прогонки тестов используется ``docker-compose.test.yml`` (упрощенная инфраструктура и запуск ``backend`` в режиме ``--test``).

5.2 Подключение приложения

Конфигурация передается через переменные окружения и env-файлы (``env/*.env``), ключевые параметры:

- ``DATABASE_DSN`` (PostgreSQL async DSN),
- ``REDIS_DSN``,
- ``RABBITMQ_DSN``,
- параметры S3/MinIO: ``AWS_ACCESS_KEY``, ``AWS_SECRET_KEY``, ``AWS_ENDPOINT_URL``, ``AWS_BUCKET_NAME``.

5.3 Миграции

Схема БД поддерживается через Alembic (скрипты в ``project/app/migrations``).

При старте ``project/main.py`` выполняется:

``alembic upgrade head``, что синхронизирует БД с текущей версией моделей.

5.4 Начальные данные

Базовые таблицы и связи создаются миграцией ``initial_migration`` (Alembic revision ``14b89c06664a``).

В рамках текущей реализации инициализация данных при старте выполняется через ``DataLoader`` в lifespan приложения backend (подготовка зависимостей и загрузка данных в окружении).

6. Реализация REST API

6.1 Точка входа

Запуск сервисов осуществляется командой вида:

- ``python main.py backend --run|--dev|--test``

- ``python main.py ml --run|--dev``
- ``python main.py scheduler --run|--dev``

В backend приложение монтирует API на префикс ``/api/v1`` (через ``app.mount("/api/v1", api_app)``), а API-роутеры подключаются в ``backend/apps/api_app.py``.

6.2 Основные модули API

В API реализованы роутеры (группы эндпоинтов) с базовыми префиксами:

- ``auth`` — ``/auth/login``, ``/auth/login/verify/code``, ``/auth/refresh``;
- ``users`` — ``/users/me``, ``/users/patch``, ``/users/registrate``, ``/users/registrate/verify/code``;
- ``admin`` — административные операции (работа с пользователями/чатами/файлами/привязками зданий и сервисов);
- ``attachments`` — загрузка/создание вложений;
- ``chats``, ``messages`` — работа с чатами и сообщениями;
- ``services`` — доступ к сервисам и аналитическим данным (`dashboard/finance/attendance/exploitation` и т.д.);
- ``buildings``, ``renters`` — управление доменными сущностями;
- ``ws`` — websocket-эндпоинт для realtime-взаимодействия.

Также доступен технический эндпоинт:

- ``GET /health`` (в рамках ``/api/v1``) для проверки статуса сервиса.

6.3 Аутентификация и разграничение доступа

Доступ к защищенным эндпоинтам реализован через ****HTTP Bearer**** (заголовок ``Authorization: Bearer <access_token>``).

Выдача токенов построена на JWT:

- ``/api/v1/auth/login`` — инициирует вход и отправку кода подтверждения,
- ``/api/v1/auth/login/verify/code`` — проверка кода и получение пары ``access_token``/``refresh_token``,
- ``/api/v1/auth/refresh`` — обновление пары токенов по ``refresh_token``.

Refresh-токены дополнительно контролируются через Redis (хранение ``jti`` refresh-токена), что позволяет отзывать/перевыпускать refresh при обновлении.

6.4 Контракты и обработка ошибок

Контракты запросов/ответов описаны Pydantic-схемами (в т.ч. ``project/app/templates/schemas/...`` и ``project/backend/templates/schemas/...``).

Ошибки приводятся к предсказуемым HTTP-ответам через общий namespace шаблонов ответов (``TemplatesResponsesDataclass`` / ``TemplateResponsesNamespace``) и middleware обработки ошибок.

6.5 Документация

FastAPI формирует OpenAPI схему автоматически. Документация доступна на стандартных эндпоинтах, смонтированных под ``/api/v1`` (например, ``/api/v1/docs``), при условии что документация не отключена конфигурацией окружения.

7. Порядок запуска проекта (локально)

7.1 Контейнерный запуск

Из корня репозитория:

- поднять окружение и пересобрать образы: ``make reload``

После запуска проверить:

- ``GET http://127.0.0.1:8000/api/v1/health`` (должно вернуть ``{"status":"ok"}``),
- доступность API через gateway при соответствующей конфигурации домена.

7.2 Запуск сервисов вручную (без compose для backend, при наличии внешней инфраструктуры)

1) Поднять зависимости (postgres/redis/rabbitmq/minio) любым способом.

2) Запустить backend:

- ``python project/main.py backend --dev`` (с hot-reload) или ``--run`` (без reload)

3) Аналогично при необходимости:

- ``python project/main.py ml --dev|--run``
- ``python project/main.py scheduler --dev|--run``

8. Результаты

- Описана и развернута инфраструктура проекта через Docker Compose (PostgreSQL, Redis, RabbitMQ, MinIO и gateway).

- Реализована реляционная схема данных для домена зданий/арендаторов/договоров/биллинга и коммуникаций (чаты/сообщения/вложения) и обеспечена ее эволюция через Alembic.
- Реализован REST API с версионированием `/api/v1` и healthcheck эндпоинтом.
- Настроена аутентификация JWT с подтверждением входа кодом и механизмом refresh-токенов с контролем в Redis.
- Подготовлен тестовый контур: docker-compose для тестов и интеграционные тесты/фикстуры (pytest + httpx ASGITransport) для ключевых потоков.

9. Выводы

В ходе работы отработан практический цикл backend-разработки на реальном проекте Pulse Project: от инфраструктуры и схемы данных до работающего API, механизма аутентификации и тестового окружения. Использование FastAPI ускорило построение HTTP-контрактов, а связка SQLAlchemy + Alembic обеспечила управляемое развитие схемы БД и воспроизводимость окружения.